

JavaScript

Chapitre 04

programmation modulaire

Introduction à la création de modules en Javascript

programmation modulaire

Préambule

Ce chapitre expose les différentes approches permettant la création de module en Javascript.

Ce chapitre s'appuie sur les références suivantes :

[https://addyosmani.com/resources/](https://addyosmani.com/resources/essentialjsdesignpatterns/book/)

[essentialjsdesignpatterns/book/](https://auth0.com/blog/javascript-module-systems-showdown/)

<https://auth0.com/blog/javascript-module-systems-showdown/>

<https://tech.mozfr.org/post/2015/08/21/ES6-en-details-%3A-les-modules>

<https://github.com/amdjs/amdjs-api/wiki/AMD>

<http://requirejs.org/>

Pour l'essentiel, les exemples sont issus du livre de **Addy Osmani** intitulé "Learning JavaScript Design Patterns".

Des modules, pour quoi faire ?

C'est avant tout une question de **conflits** et de **dépendances**.

L'encapsulation permet d'éviter les conflits et facilite le développement.

La gestion des dépendances reste à la charge du développeur mais ce point peut être en partie amélioré.

Définir et charger un module en Javascript

programmation modulaire

Le module Ad-hoc

Avec les concepts vu dans les chapitres précédents, il est déjà possible d'écrire quelque chose qui se rapproche d'un module Javascript.

L'exemple à droite est celui du *RevealingModule Pattern* introduit par Christian Heilmann.

Il existe beaucoup d'autres variations présentant des avantages/inconvénients différents.

```
var myRevealingModule = (function () {  
    var privateVar = "Ben Cherry",  
        publicVar = "Hey there!";  
  
    function privateFunction() {  
        console.log( "Name:" + privateVar );  
    }  
  
    function publicSetName( strName ) {  
        privateVar = strName;  
    }  
  
    function publicGetName() {  
        privateFunction();  
    }  
  
    // Reveal public pointers to  
    // private functions and properties  
    return {  
        setName: publicSetName,  
        greeting: publicVar,  
        getName: publicGetName  
    };  
})();  
  
myRevealingModule.setName( "Paul Kinlan" );
```

Module ad-hoc

avantages

- ✓ C'est simple à programmer
- ✓ L'encapsulation du code est bonne
- ✓ On a une vision claire de la portée des variables et des méthodes

inconvénients

- ✗ On ne peut pas gérer l'importation de modules
- ✗ Les dépendances doivent être gérées à la main
- ✗ Le module est chargé de manière synchrone

CommonJS

L'objectif du projet *CommonJS* était de définir un ensemble de spécifications pour aider le développement d'applications Javascript coté serveur. Les modules NodeJS s'inspirent largement de ses recommandations sans toutefois les suivre exactement.

<http://requirejs.org/docs/commonjs.html>

Le code d'un module CommonJS classique ressemble à ce qui suit :

```
// In circle.js
const PI = Math.PI;
exports.area = (r) => PI * r * r;
exports.circumference = (r) => 2 * PI * r;
```

L'import du module est réalisé par la fonction **require**

```
// In some file
const circle = require('./circle.js');
console.log( `The area of a circle of radius 4 is
${circle.area(4)}` );
```

CommonJS

avantages

- ✓ Reste simple à programmer.
- ✓ Il est possible de gérer les dépendances. Un module peut **require** un autre module.

inconvénients

- ✗ Le chargement des modules est forcément synchrone. Il suit les appels à **require**.
- ✗ Un fichier = un module.
- ✗ Il faut une bibliothèque pour l'utiliser dans un navigateur.

Asynchronous Module Definition (AMD)

Contrairement à **CommonJS**, **AMD** se focalise sur la définition de modules utilisables dans le navigateur. Son API spécifie un mécanisme qui permet de définir les modules et leurs dépendances de telle sorte qu'ils puissent être chargés de manière asynchrone. La spécification repose sur la définition d'une seule fonction :

```
define(id?, dependencies?, factory);
```

aller plus loin : <https://github.com/amdjs/amdjs-api>
Exemple de définition d'un module

```
//my/shirt.js now has some dependencies, a cart and
inventory
//module in the same directory as shirt.js
define("foo/title",
    ["/cart", "/inventory"],
    function(cart, inventory) {
        //return an object to define the "my/shirt"
module.
        return {
            color: "blue",
            size: "large",
            addToCart: function() {
                inventory.decrement(this);
                cart.add(this);
            }
        }
    }
);
```

Et de son chargement :

```
require(["/cart", "/inventory"], function ( cart,
inventory ) {
    // rest of your code here
    cart.doSomething();
});
```

AMD

avantages

- ✓ Une évaluation en deux passes qui permet le chargement asynchrone des modules et de leur dépendance.
- ✓ Compatible avec **require** et **exports**.
- ✓ Il est possible de gérer les dépendances.

inconvénients

- ✗ Plus complexe à écrire.
- ✗ Il faut une bibliothèque de chargement pour l'utiliser dans un navigateur (requireJS, CURL, etc.).

Universal Module Definition (UMD)

AMD : Approche basée sur l'utilisation de modules dans le navigateur.

CommonJS : Approche basée sur l'utilisation de modules côté serveur.

UMD : essai de combiner les deux.

Exemple

Définition d'un module commonJsStrict qui dépend d'un module **b** :

```
(function ( root, factory ) {  
  if ( typeof exports === 'object' ) {  
    // CommonJS  
    factory( exports, require('b') );  
  } else if ( typeof define === 'function' &&  
define.amd ) {  
    // AMD. Register as an anonymous module.  
    define( ['exports', 'b'], factory);  
  } else {  
    // Browser globals  
    factory( (root.commonJsStrict = {}), root.b );  
  }  
})(this, function ( exports, b ) {  
  //use b in some fashion.  
  
  // attach properties to the exports object to  
define  
  // the exported module properties.  
  exports.action = function () {};  
}));
```

aller plus loin : <https://github.com/umdjs/umd/tree/>

master/templates

Modules ES2015

ES2015 introduit dans la norme le support des modules. Aujourd'hui la majorité des runtime prennent en charge cette fonctionnalité bien que quelques exceptions demeurent.

Exemple

Définition d'un module en ES2015 :

```
//----- lib.js -----  
export const sqrt = Math.sqrt;  
export function square(x) {  
    return x * x;  
}  
export function diag(x, y) {  
    return sqrt(square(x) + square(y));  
}
```

Et son utilisation (chargement) :

```
//----- main.js -----  
import { square, diag } from 'lib';  
console.log(square(11)); // 121  
console.log(diag(4, 3)); // 5
```

aller plus loin : <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules>

Module ES2015

avantages

- ✓ Chargement synchrone et asynchrone (par défaut).
- ✓ Intégré au langage.
- ✓ Gestion des dépendances.

inconvénients

- ✗ Pas forcément disponible partout. Mais peut être utilisé via un *transpileur* comme *Babel*.
- ✗ Problème d'utilisation sur certain serveur à cause du type mime (`application/javascript`) et de l'extension `.mjs` pas toujours bien reconnu (mais il existe des solutions).

Empaqueteur de code

programmation modulaire

Quésaco ?

Lorsque l'on parle d'empaqueteur de code (Module bundler, en anglais), de quoi parle-t-on ?

Un empaqueteur de code est un programme qui à partir d'un **ensemble de modules et de leurs dépendances**, va produire un nouveau module qui se suffira à lui-même.

En fonction du type d'empaqueteur utilisé, ce module peut être constitué d'un **fichier unique (bundle)** ou bien constituer le point d'entrée vers **différents morceaux (chunks)** qui seront chargés dynamiquement.

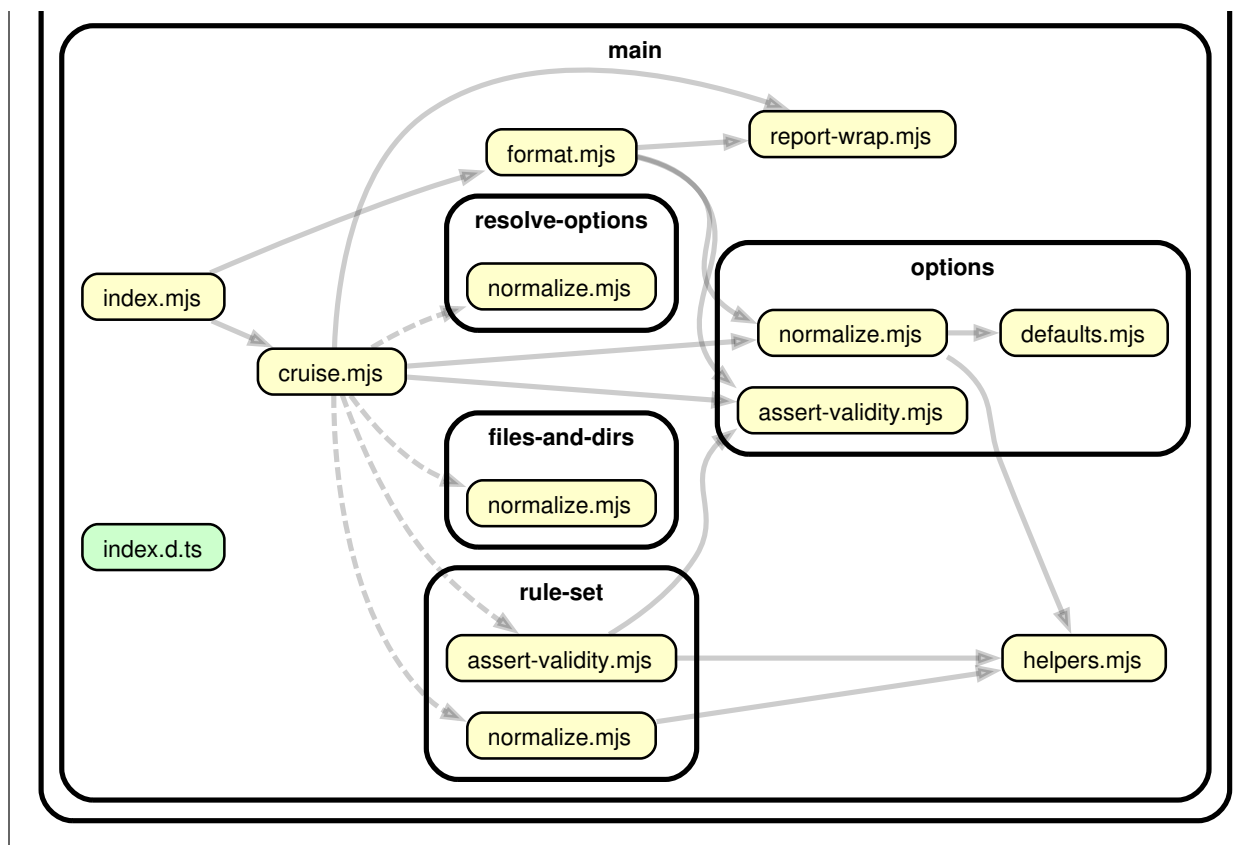
La majorité des empaqueteurs intègrent des processus d'optimisations qui vont permettre de générer un package adapté à une cible particulière. On va par exemple pouvoir viser une version précise d'un navigateur web.

```
{ "browserslist": "> 0.25%, not dead" }
```

Aller plus loin :

<https://www.codingspark.io/glossary/modules-bundler>

<https://kinsta.com/fr/blog/rollup-vs-webpack-vs-parcel/>



Graphe de dépendance de **dependency-cruiser**
(cf. <https://github.com/sverweij/dependency-cruiser/tree/main>)

Webpack

<https://webpack.js.org/>

Webpack est un empaqueteur de code JavaScript utilisé pour gérer des applications complexes. Il permet de regrouper des fichiers, de faire du *code splitting* et propose une large gamme de plugins pour personnaliser le processus de compilation.

POUR ET CONTRE

- ✓ Très flexible
- ✓ Découpage du code (Code splitting)
- ✓ Large écosystème
- ✗ Potentiellement complexe à configurer
- ✗ Temps de build long

USAGE

Webpack est un bon choix pour développer des applications complexes qui nécessiteraient un empaqueteur flexible.

Rollup

<https://rollupjs.org/>

Rollup est conçu pour optimiser les bibliothèques JavaScript en supprimant le code inutilisé (tree shaking). Il est particulièrement efficace pour la création de modules légers nécessitant différents formats de sortie (ESM, CommonJS, etc.).

POUR ET CONTRE

- ✓ Élimination du code inutilisé (Tree shaking) efficace
- ✓ Nombreux formats de sortie (CommonJS, ESM, UMD, ...)
- ✓ Configuration moins verbeuse que Webpack
- ✗ Moins performant sur les applications complexes
- ✗ Moins flexible que Webpack
- ✗ Écosystème plus réduit

USAGE

Rollup est bien adapté au développement de bibliothèques JavaScript et des projets nécessitant une élimination de code poussée.

Parcel

<https://parceljs.org/>

Parcel est un empaqueteur de code qui fonctionne immédiatement. Il prend en charge de nombreux types de fichiers (JS, CSS, HTML) sans nécessiter de fichiers de configuration. Il est idéal pour les petits projets ou les prototypes.

POUR ET CONTRE

- ✓ Ne nécessite presque aucune configuration
- ✓ Très rapide pour les petits projets
- ✓ Support multi-langage (JavaScript, TypeScript, CSS, HTML, ...) sans configuration supplémentaire
- ✗ Pas aussi optimisé que Webpack ou Rollup
- ✗ Moins configurable et extensible que Webpack.

USAGE

Parcel est idéal pour des projets simples ou le développement de prototypes.

ViteJS

<https://vitejs.dev/>

POUR ET CONTRE

- ✓ Très rapide en développement comme en production puisqu'il utilise des modules ES natifs
- ✓ Simple à configurer comme Parcel mais en restant très flexible
- ✗ Écosystème encore assez pauvre du car récent
- ✗ Production de build moins optimisé que Rollup

USAGE

Vite est un excellent choix pour développer des projets modernes utilisant des ES natifs

Webpack

programmation modulaire

Introduction

Qu'est-ce que *webpack* ?

Webpack permet de transformer, regrouper ou emballer différentes sources pour produire des modules. Il présente l'avantage d'unifier l'utilisation des différents formats de modules vus précédemment : AMD, CommonJS, UMD, ES2015.

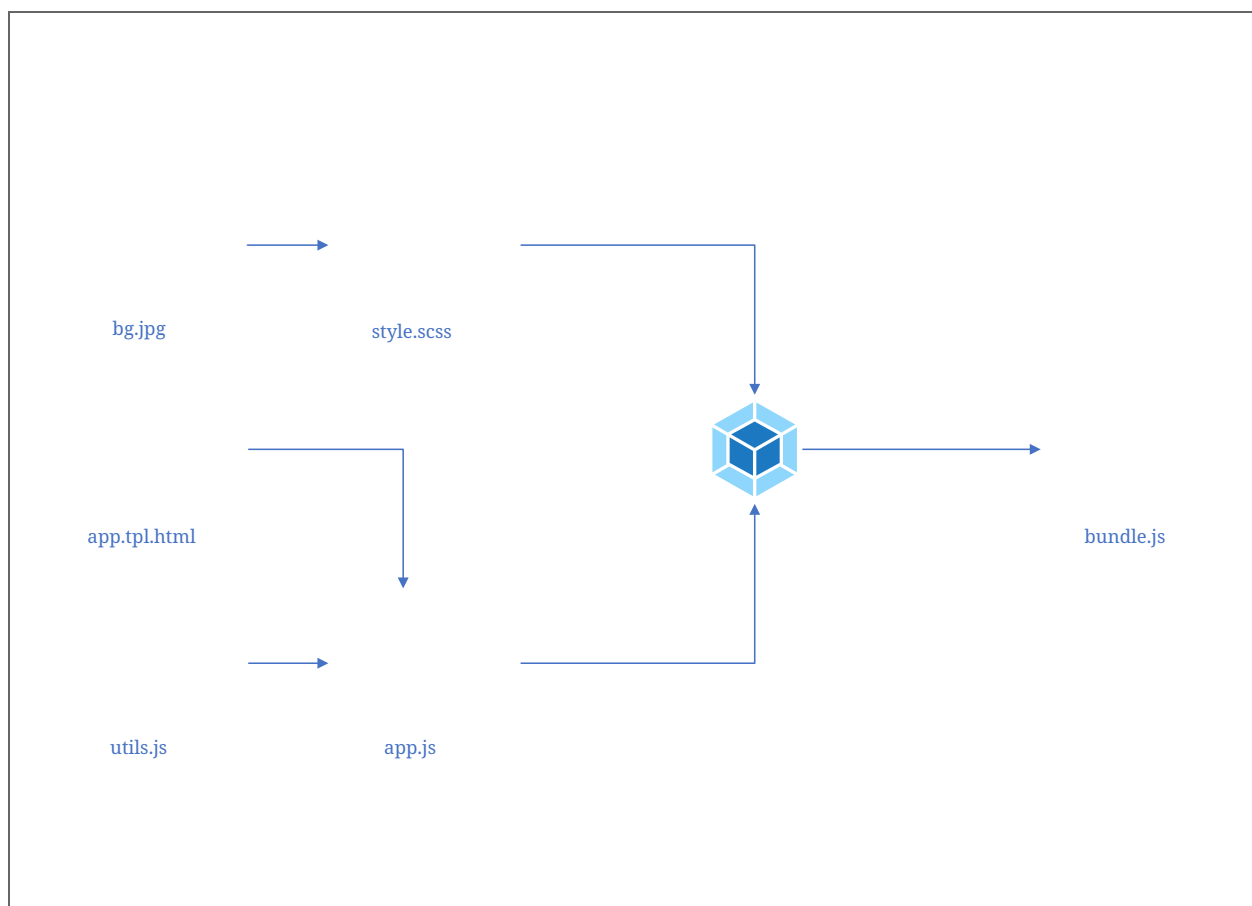
Nativement, webpack supporte uniquement les fichiers *JavaScript* et *JSON* mais via l'utilisation de plugins, il est possible d'utiliser d'autres types de sources comme des fichiers html, css, des images, etc.

aller plus loin : **<https://webpack.js.org/guides/getting-started/>**

Webpack et les dépendances

webpack permet ainsi d'interpréter chaque partie de l'application comme s'il s'agissait d'un module que ce soit du javascript, une feuille de style (css ou même scss), une image, etc.

En permettant de manipuler différentes sources comme s'il s'agissait de module, webpack permet de gérer proprement la notion de dépendances.



Fonctionnement de webpack (source : <https://webpack.js.org/>)

Configuration de webpack

La configuration de webpack repose sur les concepts qui suivent (cf. <https://webpack.js.org/concepts/>).

Entry

Les points d'entrées par lesquels webpack doit commencer pour construire la liste des dépendances.

```
entry: {  
  app: './index.js',  
  styles: './stylesheets/styles.scss'  
}
```

Output

Les endroits et les noms à utiliser pour produire les paquets.

```
output: {  
  path: path.resolve(__dirname, './public'),  
  filename: 'assets/js/[name].js',  
  library: 'myApp',  
  libraryTarget: 'umd'  
}
```

Loaders

Les méthodes et les règles à utiliser pour transformer et identifier les fichiers sources (par défaut webpack ne traite que les fichiers js et les fichier json).

```
module: {  
  rules: [  
    ...  
    {  
      test: /\. (js|jsx)$/ ,  
      exclude: /node_modules/ ,  
      use: [{  
        loader: 'babel-loader' ,  
        options: {  
          presets: [  
            ['@babel/preset-env' , { 'targets': {  
'browsers': '> .5% or last 3 versions' } }]  
          ]  
        }  
      }]  
    }  
  ]  
}
```

Plugins

Les méthodes à appliquer pour réaliser différentes tâches lors du développement ou de la mise en production. Par exemple :

- minimiser une fichier js
- étendre la compatibilité des css
- construire un fichier html à partie d'un gabarit
- etc.

```
plugins: [  
  ...  
  new CleanWebpackPlugin(['public']),  
  new MiniCssExtractPlugin({  
    filename: 'assets/stylesheets/[name].css',  
    chunkFilename: '[id].css'  
  })  
  ...  
]
```

Exemple d'utilisation

Création des fichiers sources

```
import bar from './bar.js';  
bar();
```

src/index.js

```
export default function bar() {  
  //  
}
```

src/bar.js

```
const path = require('path');  
  
module.exports = {  
  entry: './src/index.js',  
  output: {  
    path: path.resolve(__dirname, 'dist'),  
    filename: 'bundle.js',  
  },  
};
```

webpack.config.js

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    ...
  </head>
  <body>
    ...
    <script src="dist/bundle.js"></script>
  </body>
</html>
```

page.html

Appel de webpack pour créer le package

```
c:\testwebpack\npx webpack
asset bundle.js 42 bytes [compared for emit]
[minimized] (name: main)
orphan modules 59 bytes [orphan] 1 module
./src/index.js + 1 modules 96 bytes [built] [code
generated]
```

Linkographie

En plus de la documentation officielle :

- **Webpack**
- **Concepts**
- **Manually Bundling an Application**
- **Live Coding a Simple Module Bundler**
- **Detailed Explanation of a Simple Module Bundler**
- **Getting-started**

Je vous conseille également la lecture suivante :

<https://www.axopen.com/blog/2021/10/webpack-projet-front-end-tuto/>

Speaker notes